2026

SHAHEED HIMAYUN MUZZAMIL MEMORIAL
GOVERNMENT DEGREE COLLEGE ANANTNAG

Department of Computer Applications

Project Report (e-yantra)

Wall Following Robot with PID Control

Batch 2022 (NEP)

Aqib Javid Bhat	65274-ANG-2022
Mohd Numan Asgar	65301-ANG-2022
Danish Ellahi	65277-ANG-2022
Ubaid Mushtaq Mir	65291-ANG-2022
Wasiq Tariq	63848-ANG-2021
Ifham Mushtaq	65292-ANG-2022

Under the Supervision of
Dr. Tawseef Ahmed Teli
Asst. Prof. Department of Computer Applications

Contents

Abstract 2
1. Introduction 2
1.1 Background and Motivation 2
1.2 Problem Statement 2
1.3 Scope and Applications 3
2. Methodology 3
2.1 Working Principle 3
2.2 Hardware Components Used 4
2.2.1 Arduino UNO 4
2.2.2 Ultrasonic Sensor. 5
2.2.3 L298N Motor Driver 5
2.2.4 Dual Shaft BO Motors 6
2.2.5 65mm BO Motor Wheels 6
2.2.6 Trolley Wheel / Caster Wheel 6
2.2.7 2WD Chassis 7
2.2.8 170-Ponit Mini Breadboard 7
2.2.9 Jumper Wires 8
2.2.10 18650X2 Battery Holder 8
2.2.11 18650 Lithium-Ion Rechargeable Batteries 8
2.2.12 ON/OFF Switch 9
2.2.13 AlphaBot2-AR Platform 9
2.3 Software and Algorithms Used 10
2.3.1 Programming Language and Platform Used 10
2.3.2 Libraries Used 10
2.3.3 Algorithm and Pseudocode 10
2.3.4 Flowchart 14
2.3.5 Code 15
3. Implementation 20
3.1 Build and Assembly 20
3.2 Challenges faced and how they were resolved 21
4. Testing and Results 22
4.1 Tests Performed 22
4.2 Observations and Outcomes 23
5. Conclusion and Future Scope 24
6. GitHub Repository 24

Abstract

Autonomous navigation in confined or maze-like environments poses significant challenges for mobile robots, particularly in maintaining consistent proximity to guiding structures like walls without oscillations or collisions. This project introduces a Wall-Following Robot utilizing PID (Proportional-Integral-Derivative) control, enhancing traditional bang-bang controllers by providing smoother, more stable path adjustment through ultrasonic sensor feedback. Constructed on an Arduino Uno-based platform with differential drive motors and 2-3 ultrasonic sensors for distance measurement, the robot dynamically computes steering corrections to sustain a predefined safe distance from the wall. It employs in-place turns for 90-degree corners and gradual differential steering for curves, reducing error accumulation and improving reliability. Implementations support both left-wall and right-wall following, adaptable to platforms like AlphaBot2-Ar or custom chassis. Experimental tuning of PID gains ensures oscillation-free motion, with demonstrations validating precise distance maintenance and efficient corner navigation. This cost-effective design integrates sensor fusion, feedback control, and real-time decision-making, offering a scalable prototype for maze-solving and exploratory robotics. Future extensions could incorporate multi-sensor fusion, adaptive PID tuning via machine learning, or integration with SLAM for mapping unknown spaces. Applications span search-and-rescue operations, warehouse inventory, and educational tools for control systems.

1. Introduction

1.1 Background and Motivation

The preceding project on a bang-bang controlled wall-following robot (available at `https://github.com/aqib-m31/Wall-Following-Robot`) effectively showcased basic reactive navigation but exposed inherent limitations, such as oscillatory behavior around the setpoint distance and imprecise corner handling due to abrupt speed changes. These issues stemmed from the binary nature of bang-bang control, which fails to account for error magnitude or rate, leading to inefficient energy use and potential collisions. This observation drove the evolution toward a PID-controlled system, a cornerstone of modern control engineering that enables proportional response, integral error elimination, and derivative damping for smoother trajectories. Inspired by industrial applications like automated warehouse robots that follow walls or shelves with high precision, this design leverages ultrasonic sensors for continuous distance feedback. The goal is to develop a robust, adaptive navigation framework that transitions from reactive to predictive control, minimizing deviations and enhancing autonomy in structured environments.

1.2 Problem Statement

The core issue addressed is the instability of threshold-based (bang-bang) wall-following in dynamic or uneven environments, where sudden motor speed toggles cause vibrations, overshooting, and unreliable corner detection. This results in inconsistent path adherence, increased wear on components, and failure in prolonged operations. The technical challenge involves engineering a feedback-driven system that continuously measures wall distance, computes nuanced corrections via PID, and executes differential steering or in-place turns, all while ensuring computational efficiency on resource-constrained microcontrollers. This demands sensor

integration for accurate ranging, algorithmic tuning for stability, and hardware calibration for platform variability, ultimately yielding precise, oscillation-free navigation suitable for real-world deployment.

1.3 Scope and Applications

The scope encompasses designing, assembling, and programming a differential-drive robot capable of autonomous wall-following using PID control and ultrasonic sensors. It focuses on maintaining a constant setpoint distance (e.g., 15-20 cm) from the wall during forward motion, with adaptations for left- or right-side following. The system handles straight segments via gradual steering adjustments and 90-degree corners via in-place rotations, assuming flat floors and predefined initial positioning. Limitations exclude dynamic obstacle avoidance beyond walls or mapping; emphasis is on control theory demonstration, sensor-motor integration, and empirical tuning for stability. This prototype serves as an educational benchmark for feedback systems, extensible to multi-wall mazes or varied terrains.

The PID-based wall-following robot finds diverse real-world applications across industries and education, demonstrating its versatility and scalability. In warehouse automation, it guides inventory robots along shelves for efficient picking and sorting, ensuring collision-free paths in narrow aisles to boost operational throughput. For search and rescue operations, the robot navigates disaster zones or collapsed structures by hugging walls, enabling efficient exploration of confined, hazardous spaces where human access is limited. Educational platforms leverage its tunable parameters to teach control systems, sensor fusion, and robotics fundamentals in STEM curricula, fostering hands-on experimentation with feedback loops and gain adjustments. In indoor cleaning or inspection tasks, it powers autonomous vacuums or drones to methodically follow room perimeters, achieving thorough coverage without manual intervention. Additionally, it excels in maze-solving competitions, reliably traversing labyrinths without external guidance to compete in robotics challenges. Beyond these, potential extensions include agricultural row-following for crop monitoring, assistive navigation in smart homes for elderly users, and pipeline inspection in industrial settings, highlighting the system's adaptability from low-cost prototypes to integrated IoT solutions for intelligent environments.

2. Methodology

2.1 Working Principle

The Wall-Following Robot with PID Control operates on a closed-loop feedback mechanism, continuously monitoring wall proximity via ultrasonic sensors to adjust motor speeds for precise path adherence. Upon initialization, the Arduino configures sensor pins and motor drivers, setting a target distance (setpoint), typically 15-20 cm, for the side-facing ultrasonic sensor, which serves as the reference for wall proximity. As the robot advances forward at a base speed, the side sensor measures the actual distance to the wall in real-time. The PID controller then computes the error as the difference between the setpoint and this measured distance. This error drives the PID algorithm: the proportional (P) term responds directly to the current error magnitude, providing a correction force proportional to deviation (e.g., larger error yields stronger steering); the integral (I) term accumulates past errors to eliminate steady-state offsets caused by constant biases like sensor drift or uneven floors; and the derivative (D) term anticipates future errors by considering the rate of change, damping oscillations to prevent overshooting. The PID gains, Kp

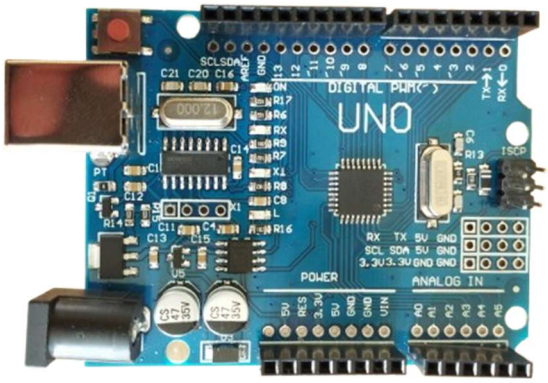
(proportional), Ki (integral), and Kd (derivative), must be empirically tuned: starting with low Kp for basic responsiveness, adding Ki sparingly to avoid windup (excessive integral buildup), and incorporating Kd for stability, often via Ziegler-Nichols methods or trial-and-error on the physical setup to balance speed, accuracy, and smoothness without instability.

The PID output modulates the motors via differential steering: by varying speeds between the left and right motors (e.g., slowing the near-side motor to turn toward the wall or speeding it up to veer away if too distant), the robot maintains the setpoint distance while progressing forward. This differential approach allows curved trajectories for gradual adjustments, contrasting with abrupt turns. For corner handling, the front sensor detects obstacles: in an "L-inside" configuration (wall ahead), the robot performs a sharp 90-degree in-place turn by reversing one motor while driving the other, reorienting to follow the new wall segment. In an "L-outside" scenario (front open but side wall lost, e.g., distance exceeds a maximum threshold), it executes a recovery curvy turn, differential steering toward the expected direction (e.g., left turn for right-wall following) until the side sensor reacquires the wall. This 'sense-compute-act' cycle repeats at high frequency, ensuring smooth curves via proportional steering and sharp, sensor-guided in-place rotations. Safety timeouts halt motion if sensors fail, promoting reliability. By replacing bang-bang's discrete actions with continuous PID modulation, the system achieves stable, energy-efficient navigation in wall-guided mazes, with the controller actively compensating for environmental variations to sustain the setpoint.

2.2 Hardware Components Used

The hardware for this project utilized standard components assembled into two distinct platforms for testing: the initial implementation was tested on the AlphaBot2-AR kit for right-wall following, providing a pre-integrated chassis with built-in sensor mounts and motor drivers. Subsequently, a fully custom robot was built from scratch to validate the left-wall following variant, allowing for tailored adjustments in sensor positioning and chassis design. This section details the key electronic and mechanical elements.

2.2.1 Arduino UNO



The Arduino Uno is a microcontroller board based on the ATmega328P. It features 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz quartz crystal, USB connection, power jack, ICSP header, and a reset button. It operates at 5V and can be powered via USB or external power supply.

In this project, the Arduino Uno acts as the central control unit. It reads ultrasonic sensor data, executes PID calculations, and outputs PWM signals to the motor driver for speed/direction control, enabling real-time path adjustments.

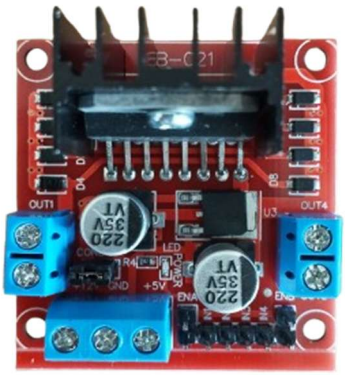
2.2.2 Ultrasonic Sensor.



The HC-SR04 ultrasonic sensor consists of a transmitter and a receiver. It operates on a 5V DC supply and measures distance using sound waves at 40kHz. It has a detection range from 2 cm to 500 cm with an accuracy of about 3 mm. The sensor sends out an ultrasonic pulse and measures the time taken for the echo to return.

For right-wall following on AlphaBot2-AR, three sensors (front, right side, and optional left for left wall following) are used. For left-wall following on the custom robot, two (front and left side). Side sensors track wall distance for PID input, while the front detects obstacles. Mounted on chassis brackets (forward-angled for corners), they interface via trigger/echo pins, providing continuous feedback.

2.2.3 L298N Motor Driver



The L298N motor driver module is a dual H-bridge driver that can control two DC motors independently. It can handle up to 2A of current per channel and supports voltage inputs from 5V to 12V. It has onboard terminal blocks for motors and power, and input pins for control signals from the Arduino.

It bridges the Arduino's low-power signals to high-current motor demands, facilitating differential steering: varying left/right speeds for PID corrections and turns while supporting forward/reverse for in-place rotations.

2.2.4 Dual Shaft BO Motors



Each BO motor operates on 3V–12V DC and provides moderate torque with low RPM, typically around 100 RPM at 6V. The dual shaft allows for attachments on both ends, useful for future upgrades like encoders for speed sensing.

These motors drive the robot's wheels and are responsible for its mobility. Their torque is suitable for lightweight robotic platforms, allowing smooth motion over flat indoor surfaces. The dual-shaft design also offers flexibility for expansion.

2.2.5 65mm BO Motor Wheels



These wheels have a diameter of 65mm and are made of lightweight plastic with a rubber outer ring for better grip. They are designed to be press-fitted onto BO motor shafts.

The wheels enable the robot to move forward, backward, and turn efficiently. Their size provides a good balance between speed and stability, making them suitable for indoor obstacle-avoiding applications.

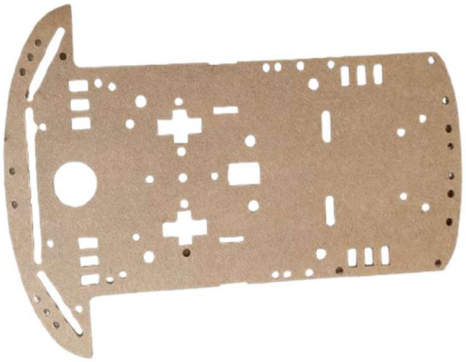
2.2.6 Trolley Wheel / Caster Wheel



The caster wheel is a freely rotating wheel that can swivel in 360 degrees. It is small in size, usually made of plastic or metal, and is mounted on a metal bracket.

This wheel supports the front end of the robot, maintaining balance while allowing free movement in all directions. It complements the two driven wheels, enabling smooth turns and preventing the robot from tipping.

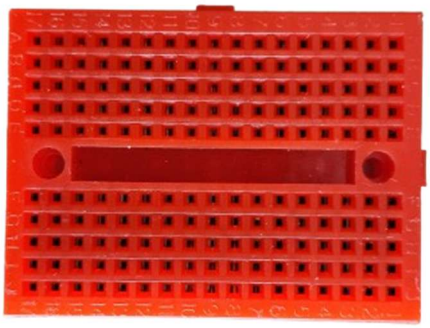
2.2.7 2WD Chassis



The 2-wheel drive chassis is typically made from acrylic or plastic and includes mounting holes for motors, battery holders, and control boards. It is lightweight, compact, and designed for BO motor compatibility.

The chassis provides the structural base for mounting all the components. It organizes the robot into a stable form factor and ensures that everything stays securely in place during movement and testing.

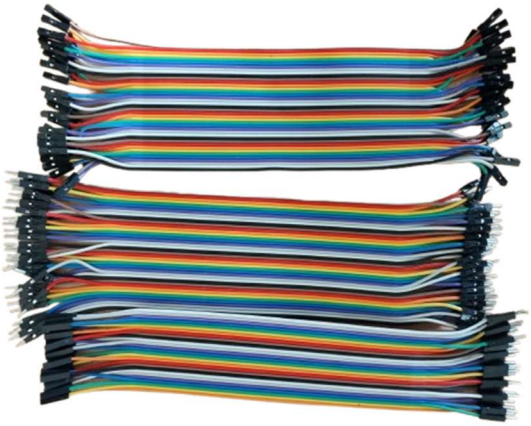
2.2.8 170-Ponit Mini Breadboard



This is a compact solderless breadboard with 170 tie-points. It is made of ABS plastic and features two power rails along with connecting strips for temporary circuit construction.

The breadboard is used to connect components like sensors, modules, and wires during the prototyping phase. It helps in testing and modifying the circuit quickly without soldering, which is ideal for student projects.

2.2.9 Jumper Wires



Jumper wires are insulated electrical wires with connector pins at each end. They come in various types such as male-to-male, male-to-female, and female-to-female, and are typically used on breadboards and headers. These wires are available in different lengths and colors for easy circuit tracing.

Jumper wires are used for making electrical connections between the Arduino, sensors, LCD, motor driver, and breadboard. They enable flexible and temporary wiring, allowing quick prototyping and easy troubleshooting without the need for soldering. They also help in organizing the connections and keeping the circuit modular.

2.2.10 18650X2 Battery Holder



The 18650X2 battery holder accommodates two 18650 lithium-ion batteries. It supplies power to the motors and the Arduino via the motor driver.

2.2.11 18650 Lithium-Ion Rechargeable Batteries



In this project, Hongli 2000mAh 3.7V 18650 Li-Ion Cell Batteries are used as high-capacity rechargeable power sources, providing approximately 3.7V each with a 2000mAh rating for sustained runtime. Two such batteries, connected in series via the 18650X2 holder, deliver around 7.4V to power the L298N motor driver, which in turn supplies the Arduino, motors, and other circuits of the robot.

2.2.12 ON/OFF Switch



The On/Off switch is a simple SPST (Single Pole Single Throw) toggle switch. It usually has two terminals and is designed to interrupt or complete a circuit, allowing current to flow only when in the "ON" position.

The switch is used to conveniently power the robot on or off without disconnecting the batteries. It adds a layer of safety and ease of use by allowing quick shutdowns during testing or in case of malfunction and helps conserve battery life when the robot is idle.

2.2.13 AlphaBot2-AR Platform



The AlphaBot2-AR is a compact, integrated mobile robotics platform built for rapid prototyping and control system testing. It features a two-wheel differential drive, front caster wheel, and onboard L298P motor drivers controllable via Arduino Uno. Pre-aligned sensor mounts and expansion headers simplify wiring and assembly, making it easy to integrate front and side ultrasonic sensors for distance measurement and navigation.

In this project, AlphaBot2-AR was used as the baseline platform for right-wall following, enabling quick setup, stable motion, and consistent results without custom fabrication. Its dual BO motors, clean power distribution, and modular chassis provided a reliable testbed for PID tuning, sensor calibration, and real-time feedback control. This allowed the team to focus on refining control

logic before transitioning to a custom-built robot for left-wall implementation.

2.3 Software and Algorithms Used

This section outlines the programming tools, logic, and flow that control the behavior of the robot.

2.3.1 Programming Language and Platform Used

The entire robot is programmed using the Arduino IDE, which is a user-friendly open-source platform for writing and uploading code to the Arduino Uno. The language used is C/C++, specifically tailored for embedded systems programming. The Arduino IDE offers built-in functions for controlling digital and analog pins, reading sensor data, and communicating with external devices, which made it easy to implement the logic required for this project.

2.3.2 Libraries Used

Libraries in Arduino are pre-written code that simplify programming by allowing users to interact with hardware using simple functions instead of dealing with complex code. To help build the robot and make the development process smoother, the NewPing.h library was used.

The *NewPing.h* library is specifically used for working with ultrasonic sensors, which are used to measure distance. It makes controlling ultrasonic sensors much easier by providing easy-to-use functions for triggering and receiving echoes. In this project, it was used to manage the front, left, and right ultrasonic sensors, enabling reliable and efficient distance measurement for PID-based wall following. This reduced code complexity and improved sensor response time, ensuring smoother real-time control.

2.3.3 Algorithm and Pseudocode

The algorithm for the PID Wall-Following Robot enables autonomous navigation along walls using ultrasonic-based distance feedback. The robot starts at a known position and continuously adjusts its path to maintain the setpoint distance. It uses ultrasonic sensors to detect wall proximity for PID corrections and executes steering movements at each cycle to maintain accuracy. The system includes predefined thresholds for turns and recovery maneuvers, ensuring stable motion even in structured environments without dynamic obstacles.

The speed thresholds (base_speed, min_speed, max_speed), PID gains (Kp, Ki, Kd), turning speeds during recovery maneuvers, and safety thresholds (safe_distance_front, max_wall_distance) are both platform-dependent and environment-dependent. While the current values are optimized for AlphaBot2-AR, they should be adjusted incrementally based on your robot's motor characteristics, weight, wheel diameter, and sensor accuracy, as well as environmental factors such as surface friction, wall texture, and ambient noise affecting ultrasonic readings. Proper tuning ensures stable, smooth, and reliable navigation before deployment.

2.3.3.1 Algorithm

Objective: Implement autonomous right-wall following using PID control with three ultrasonic sensors to maintain a target distance from the right wall while avoiding obstacles.

1. Initialization Phase
    i. Configure motor control pins (PWM and direction pins for left and right motors)
    ii. Configure ultrasonic sensor pins (trigger and echo pins for front and specified wall sensor)
    iii. For right-wall following: Use right sensor; for left-wall following: Use left sensor
    iv. Initialize PID control variables: setpoint (10 cm), error, integral, last_error
    v. Set PID constants: Kp = 2.0, Ki = 0.01, Kd = 30.0
    vi. Define control parameters: base_speed, max_speed, min_speed
    vii. Set safety thresholds: safe_distance_front = 20 cm, max_wall_distance = 50 cm
    viii. Display setpoint on serial monitor
2. Sensor Reading Phase
    i. Continuously read distance values from front ultrasonic sensor
    ii. Read distance from wall-following sensor (right or left depending on implementation)
    iii. Store distances in respective variables
    iv. Output sensor readings to serial monitor for debugging
3. Front Wall Detection Phase (L-Inside Corner)
   If front sensor detects obstacle (distance < 20 cm and ≠ 0):
    i. Stop motors for 500 ms
    ii. Right-wall following: Turn left in place until wall front clears
    iii. Left-wall following: Turn right in place until wall front clears
    iv. Reset PID accumulator (integral = 0) and last error
    v. Return to sensor reading phase
4. Wall Detection Phase (L-Outside Corner/Wall Lost)
   If wall sensor reads 0 or distance > 50 cm (L-outside corner detected):
    i. Execute recovery curvy turn using differential steering
    ii. Right-wall following: Increase left motor speed (base_speed + 30), decrease right motor speed (base_speed - 20); turn right to search for wall
    iii. Left-wall following: Decrease left motor speed (base_speed - 20), increase right motor speed (base_speed + 30); turn left to search for wall
    iv. Continue forward motion while steering, creating smooth curved trajectory toward wall
    v. Reset PID variables (integral = 0, last_error = 0) to prevent integral windup
    vi. Allow robot to reacquire wall during recovery turn
    vii. Return to sensor reading phase
5. PID Wall-Following Control Phase
    i. Calculate error: error = wallDistance − setpoint
    ii. Calculate proportional term: P = error
    iii. Accumulate integral term: I += error
    iv. Calculate derivative term: D = error − last_error
    v. Compute PID output: output = (Kp × P) + (Ki × I) + (Kd × D)
6. Motor Speed Calculation Phase
   For right-wall following:
    i. Positive error (too far from wall) → reduce right motor speed (turn right)
    ii. Negative error (too close to wall) → reduce left motor speed (turn left)

iii. speedLeft = base_speed + PID_output
iv. speedRight = base_speed − PID_output

For left-wall following:

i. Positive error (too far from wall) → reduce left motor speed (turn left)
ii. Negative error (too close to wall) → reduce right motor speed (turn right)
iii. speedLeft = base_speed − PID_output
iv. speedRight = base_speed + PID_output

Constrain speeds within [min_speed, max_speed] range

Apply constrained speeds to motors

7. State Update Phase
   i. Update last_error for next PID calculation
   ii. Output debug information (error, PID output, motor speeds)
8. Loop Continuation

   Repeat sensing, detection, and control phases continuously

2.3.3.2 Pseudocode

```
BEGIN Wall-Following Robot PID Control

Initialize()

    Configure motor pins (PWMA, AIN1, AIN2, PWMB, BIN1, BIN2)

    Configure ultrasonic pins (TRIG_FRONT, ECHO_FRONT, TRIG_WALL, ECHO_WALL)

    Set setpoint ← 10 cm

    Set Kp ← 2.0, Ki ← 0.01, Kd ← 30.0

    Set base_speed ← 55, max_speed ← 65, min_speed ← 50

    Set safe_distance_front ← 20 cm, max_wall_distance ← 50 cm

    Initialize: error ← 0, integral ← 0, last_error ← 0

    Print setpoint to serial

END

WHILE TRUE

    // READ SENSORS

    distanceFront ← read front ultrasonic

    distanceWall ← read wall ultrasonic (right or left)

    Print distances to serial

    // L-INSIDE CORNER: Front wall detected

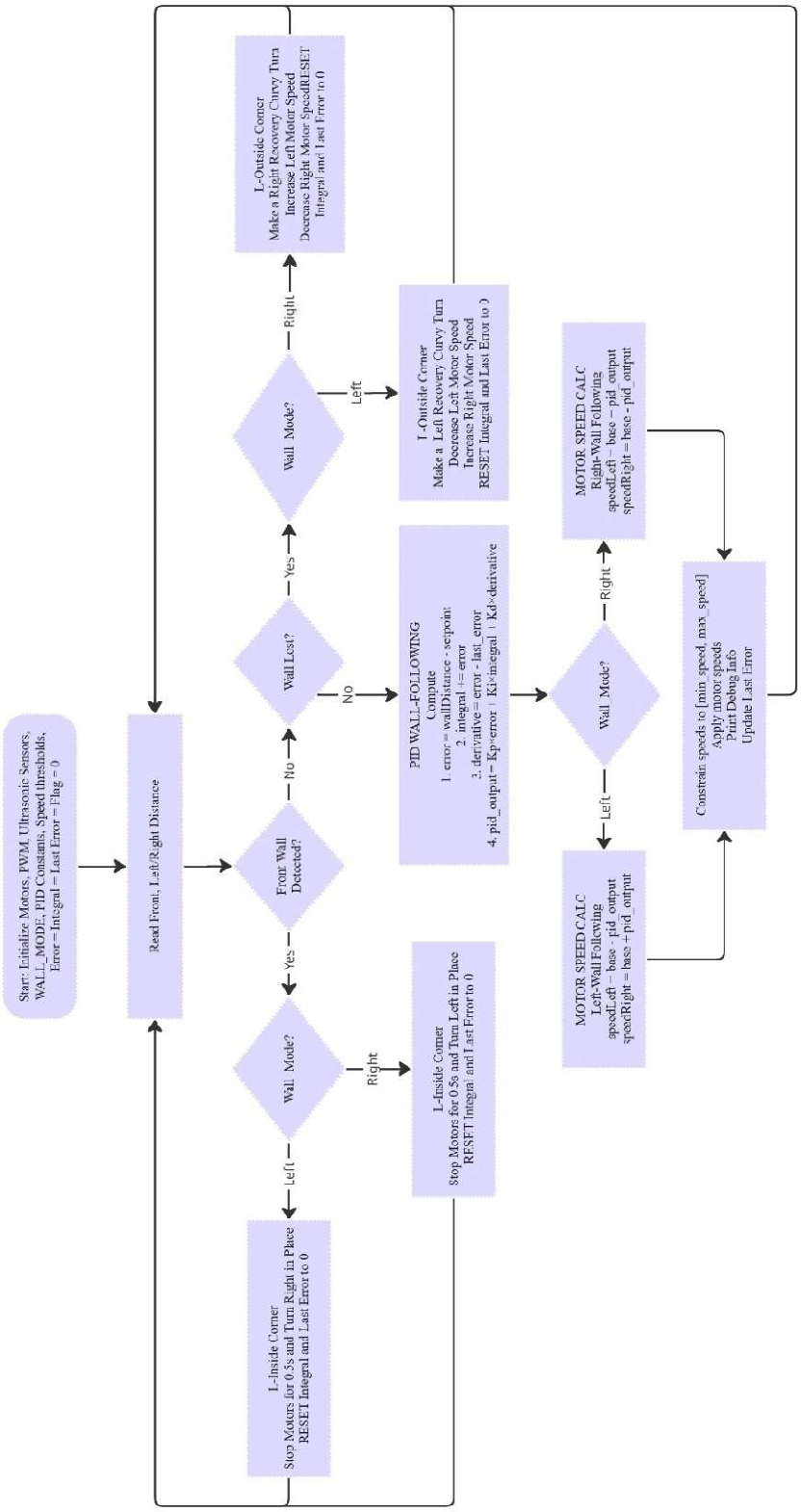
    IF distanceFront < 20 AND distanceFront ≠ 0 THEN

        Stop motors, delay 500ms

        Turn away from wall (left for right-wall, right for left-wall)
```

```
            integral ← 0, last_error ← 0
            CONTINUE
        END IF
        // L-OUTSIDE CORNER: Wall loss detected
        IF distanceWall = 0 OR distanceWall > 50 THEN
            Execute recovery curvy turn using differential steering
              Right-wall: speedLeft ← base_speed + 30, speedRight ← base_speed - 20
              Left-wall: speedLeft ← base_speed - 20, speedRight ← base_speed + 30
            integral ← 0, last_error ← 0
            CONTINUE
        END IF
        // PID WALL-FOLLOWING
        error ← distanceWall - setpoint
        integral ← integral + error
        derivative ← error - last_error
        pid_output ← (Kp × error) + (Ki × integral) + (Kd × derivative)
        // CALCULATE MOTOR SPEEDS
        IF WALL_MODE = RIGHT_WALL THEN
            speedLeft ← base_speed + pid_output
            speedRight ← base_speed - pid_output
        ELSE IF WALL_MODE = LEFT_WALL THEN
            speedLeft ← base_speed - pid_output
            speedRight ← base_speed + pid_output
        END IF
        Constrain speedLeft, speedRight to [min_speed, max_speed]
        Apply motor speeds
        // UPDATE STATE
        last_error ← error
        Print error, pid_output, motor speeds to serial
    END WHILE
END
```

2.3.4 Flowchart



2.3.5 Code

This is the right-wall following implementation, tested on the AlphaBot2-AR platform. The complete source code, along with both right and left wall-following implementations, is available in the GitHub repository: https://github.com/aqib-m31/Wall-Following-Robot-PID

```
/*
  WallFollowingRobotPID.ino
  An autonomous right-wall following robot using PID control with
three ultrasonic sensors. The robot moves forward while maintaining
a safe distance from the right wall, adjusting its path if the wall
is too close or too far.
*/

#include <NewPing.h>

// Motor Pins
#define PWMA 6   // Left Motor Speed pin (ENA)
#define AIN2 A0  // Motor-L forward (IN2)
#define AIN1 A1  // Motor-L backward (IN1)
#define PWMB 5   // Right Motor Speed pin (ENB)
#define BIN1 A2  // Motor-R forward (IN3)
#define BIN2 A3  // Motor-R backward (IN4)

// Ultrasonic Sensor Configuration
#define ECHO_FRONT 2
#define TRIG_FRONT 3
#define ECHO_LEFT 8
#define TRIG_LEFT 7
#define ECHO_RIGHT 10
#define TRIG_RIGHT 9

const int MAX_DISTANCE = 400;
NewPing sonarFront(TRIG_FRONT, ECHO_FRONT, MAX_DISTANCE);
NewPing sonarLeft(TRIG_LEFT, ECHO_LEFT, MAX_DISTANCE);
NewPing sonarRight(TRIG_RIGHT, ECHO_RIGHT, MAX_DISTANCE);

// Distance Variables
int distanceFront = 0;
int distanceLeft = 0;
int distanceRight = 0;

// PID Control Variables
```

```
int setpoint = 10;  // Desired distance from right wall (cm)
int last_error = 0;
long integral = 0;

// PID Constants (tune these for the robot)
float Kp = 2.0;   // Proportional gain
float Ki = 0.01;  // Integral gain
float Kd = 30.0;  // Derivative gain

// Motor Control
const int base_speed = 55;  // Base forward speed
const int max_speed = 65;
const int min_speed = 50;

// Safety thresholds
int safe_distance_front = 20;
int max_wall_distance = 50;  // If right sensor reads beyond this,
no wall detected, for right turn

int flag = 0;

// Move forward with differential speeds for steering
void forward(int speedLeft, int speedRight) {
  flag = 0;

  // Constrain speeds
  speedLeft = constrain(speedLeft, 0, max_speed);
  speedRight = constrain(speedRight, 0, max_speed);

  analogWrite(PWMA, speedLeft);
  analogWrite(PWMB, speedRight);
  digitalWrite(AIN1, LOW);
  digitalWrite(AIN2, HIGH);
  digitalWrite(BIN1, LOW);
  digitalWrite(BIN2, HIGH);
}

// Turn left in place
void turnLeftInPlace(int speed = 60) {
  Serial.println("Turning Left in place");
  analogWrite(PWMA, speed);
  analogWrite(PWMB, speed);
  digitalWrite(AIN1, HIGH);
```

```
  digitalWrite(AIN2, LOW);
  digitalWrite(BIN1, LOW);
  digitalWrite(BIN2, HIGH);
}

// Turn right in place
void turnRightInPlace(int speed = 60) {
  Serial.println("Turning Right in place");
  analogWrite(PWMA, speed);
  analogWrite(PWMB, speed);
  digitalWrite(AIN1, LOW);
  digitalWrite(AIN2, HIGH);
  digitalWrite(BIN1, HIGH);
  digitalWrite(BIN2, LOW);
}

// Stop motors
void stop() {
  analogWrite(PWMA, 0);
  analogWrite(PWMB, 0);
  digitalWrite(AIN1, LOW);
  digitalWrite(AIN2, LOW);
  digitalWrite(BIN1, LOW);
  digitalWrite(BIN2, LOW);
}

void setup() {
  Serial.begin(9600);

  // Set ultrasonic pins
  pinMode(ECHO_FRONT, INPUT);
  pinMode(TRIG_FRONT, OUTPUT);
  pinMode(ECHO_LEFT, INPUT);
  pinMode(TRIG_LEFT, OUTPUT);
  pinMode(ECHO_RIGHT, INPUT);
  pinMode(TRIG_RIGHT, OUTPUT);

  // Set motor pins as outputs
  pinMode(PWMA, OUTPUT);
  pinMode(AIN2, OUTPUT);
  pinMode(AIN1, OUTPUT);
  pinMode(PWMB, OUTPUT);
  pinMode(BIN1, OUTPUT);
```

```
  pinMode(BIN2, OUTPUT);

  // Initial stop
  stop();

  Serial.print("Setpoint: ");
  Serial.print(setpoint);
  Serial.println(" cm");
  delay(2000);
}

void loop() {
  // Read distance values (in cm) from each ultrasonic sensor
  distanceFront = sonarFront.ping_cm();
  distanceLeft = sonarLeft.ping_cm();
  distanceRight = sonarRight.ping_cm();

  // Print sensor readings for debugging
  Serial.print("Left: ");
  Serial.println(distanceLeft);
  Serial.print("Front: ");
  Serial.println(distanceFront);
  Serial.print("Right: ");
  Serial.println(distanceRight);

  // Wall in front - turn left
  if ((distanceFront < safe_distance_front) && (distanceFront != 0))
{
    Serial.println(" front wall");
    if (flag == 0) {
      stop();
      delay(500);
      flag = 1;
    }
    turnLeftInPlace();
    integral = 0;
    last_error = 0;
    return;
  }

  // No wall on right - turn right to find it
  if ((distanceRight == 0) || (distanceRight > max_wall_distance)) {
    Serial.println(" No wall on right, turn right to search");
```

```
    forward(base_speed + 30, base_speed - 20);  // Turn Right
    integral = 0;
    last_error = 0;
    return;
  }

  // PID wall following
  // Calculate error (positive = too far, negative = too close)
  int error = distanceRight - setpoint;

  // Calculate PID terms
  int proportional = error;
  integral += error;
  int derivative = error - last_error;

  // Calculate PID output
  float pid_output = (Kp * proportional) + (Ki * integral) + (Kd *
derivative);

  // Calculate motor speeds
  // Positive error (too far) -> turn right (slow down right motor)
  // Negative error (too close) -> turn left (slow down left motor)
  int speedLeft = base_speed + pid_output;
  int speedRight = base_speed - pid_output;

  // Ensure minimum speed for both motors
  speedLeft = constrain(speedLeft, min_speed, max_speed);
  speedRight = constrain(speedRight, min_speed, max_speed);

  // Apply motor speeds
  forward(speedLeft, speedRight);

  // Remember last error
  last_error = error;

  // Debug output
  Serial.print("Error: ");
  Serial.print(error);
  Serial.print(" PID: ");
  Serial.print(pid_output);
  Serial.print(" Left Speed: ");
  Serial.print(speedLeft);
  Serial.print(" Right Speed: ");
```

```
  Serial.println(speedRight);
}
```

3. Implementation

3.1 Build and Assembly

1. Assembling the Chassis

   Robot assembly began with assembling the 2WD (two-wheel drive) chassis. Dual shaft BO motors were attached to the frame using screws and brackets. The trolley wheel was mounted at the front to support balance and movement.

2. Mounting the Wheels

   The 65mm BO Motor Wheels were attached to the motor shafts. Motor wires, which were soldered to the BO Motors, were carefully routed to avoid tangling during motion.

3. Connecting the Motor Driver (L298N)

   Mount the Arduino UNO module and the Motor Driver (L298N) module on the top of the Chassis using a glue gun. Connect the DC motors to the L298N motor driver module using their respective motor wires. These motor wires were connected in such a way that each pair of motors from a particular side (left or right) shared the same control signals, allowing us to use a single motor driver. After that, the Power Input Pins (IN1, IN2, IN3, IN4) and Control Pins (ENA, ENB) were connected to the Arduino UNO.

4. Attaching the Ultrasonic Sensors

   Three HC-SR04 ultrasonic sensors were installed at the front side of the robot. One was attached at the center, and one each on the left and right sides of the chassis using glue gun. These sensors were then connected to specific digital pins on the Arduino for trig and echo signals.

5. Power Supply

   A 2-cell 18650 battery holder was fixed to the underside of the chassis using a glue gun. An ON/OFF switch was installed in-line with the power connection to control the robot's operation. The system was powered using two Hongli 2000mAh 3.7V 18650 Li-Ion Cell Batteries, which offered sufficient voltage and current for both the motors and control circuitry.

6. Breadboard and Wiring

   A mini breadboard, placed at the top of the chassis, was used to organize the power connections, especially for distributing VCC and GND lines to various components. Jumper wires were used to interconnect all the modules, including the Arduino and Motor Driver. To maintain structural cleanliness and prevent loose wiring from interfering with movement, cable ties were used for proper cable management.

7. Uploading the Code

   The Arduino was connected to a computer using a USB cable, and the final code was uploaded to the Arduino using the Arduino IDE. For right-wall following, initial testing was conducted on the AlphaBot2-AR platform to leverage its integrated mounts.

3.2 Challenges faced and how they were resolved

During the building and testing phase, we faced several practical challenges that tested our problem-solving skills. The issues we encountered and how we dealt with them are as follows:

1. PID Tuning Oscillations

   i. Observation: High initial Kp values caused excessive vibrations around the setpoint.

   ii. Analysis: An overly aggressive proportional response amplified small errors, leading to instability.

   iii. Mitigation: PID gains were tuned iteratively, starting with a low Kp, gradually increasing it to the oscillation threshold, then adding Kd for damping and minimal Ki to correct steady-state offsets. This stabilized the response and eliminated oscillations.

2. Corner Detection Misses

   i. Observation: The side sensor occasionally failed to detect L-outside turns, resulting in overshooting corners.

   ii. Analysis: Suboptimal sensor mounting angles and front detection thresholds delayed wall reacquisition.

   iii. Mitigation: Side sensors were mounted with a forward angle, and the front obstacle threshold was calibrated to 10 cm through repeated trials. Recovery turn durations were fine-tuned to ensure wall reacquisition within 1 second.

3. Platform Variability

   i. Observation: Motor response differed between the AlphaBot2-AR and the custom chassis, affecting base speeds and turning behavior.

   ii. Analysis: Differences in wheel friction, weight distribution, motor characteristics, motor drivers, and battery supply impacted differential steering and turn precision, requiring careful speed selections for reliable performance across platforms.

   iii. Mitigation: Speed thresholds (base, min, max) were empirically retuned for each platform. Additional traction tests minimized slippage and ensured consistent steering response.

4. Sensor Noise

   i. Observation: Ultrasonic sensor readings fluctuated on uneven floors, causing occasional spikes in PID output.

ii. Analysis: Echo interference from floor irregularities introduced inconsistent distance measurements.

iii. Mitigation: Erroneous readings of 0 were treated as infinity to prevent false wall detections, maintaining safe navigation. Testing was prioritized on flat surfaces to reduce noise effects.

5. Sensor Failure Over Time

i. Observation: The robot collided with walls after approximately 27 seconds of operation, as ultrasonic sensors returned erroneous 0 readings.

ii. Analysis: Sensor degradation, possibly due to overheating or manufacturing defects, caused false proximity detections and unsafe navigation.

iii. Mitigation: Faulty sensors were replaced immediately. Pre-testing protocols were established for all components (sensors, motors, drivers, batteries) in controlled environments, including outdoor simulations, to verify reliability under varied conditions.

4. Testing and Results

4.1 Tests Performed

To evaluate the performance of the Wall Following Robot with PID Control, a series of tests were conducted in varying indoor environments featuring both L-inside and L-outside corner turns. Right-wall following was tested on the AlphaBot2-AR platform, while left-wall following was validated on the custom robot.

1. Straight-Line Stability Test

   The robot was instructed to traverse a 1m straight wall segment multiple times. Distance variance from setpoint was measured to assess PID adherence.

2. Corner Navigation Test

   The robot approached 90° L-inside and L-outside turns 10 times each. For L-inside (front wall detected), a 90-degree in-place turn was executed after an immediate stop to prevent collision. For L-outside (side wall lost), a curvy recovery turn via differential steering was performed until wall reacquisition. Success was evaluated by collision rate, overshoot, and reacquisition time.

3. Endurance and Power-Drop Test

   The robot followed a continuous wall path to evaluate long-term stability. Battery voltage drop effects on performance were observed, particularly how reduced voltage led to gradual speed decreases impacting responsiveness.

4. Comparative vs. Bang-Bang Test

Parallel runs compared PID to the prior bang-bang system for oscillation and efficiency metrics.

4.2 Observations and Outcomes

1. Straight-Line Accuracy

   PID maintained setpoint within ±1 cm, eliminating oscillations seen in bang-bang. Differential steering ensured smooth curves and no jerky movement.

2. Corner Navigation Accuracy

   95% success rate across both L-inside and L-outside turns; in-place turns handled L-inside reliably (0.5s average after stop), while recovery turns reacquired walls in <1s for L-outside. However, in scenarios with blind spots, such as when the front ultrasonic sensor was centered and missed subtle gaps, the robot occasionally hit the wall, mistaking open space for proximity. Residual error from turn calibration was minimal.

3. Endurance

   Performance endurance varied by platform and components: the AlphaBot2-AR endured longer runs due to efficient power management, while the custom robot consumed more power from its 12V DC motors, leading to quicker voltage drops and speed reductions. As battery voltage declined, overall speeds decreased, slightly affecting turn precision but not setpoint adherence. No complete failures occurred, though dependency on battery quality and motor efficiency was evident.

Table 1 Summary of Test Results

Test Name	Success Rate	Key Observation	Issues Noted
Straight-Line Stability	98%	PID held setpoint ±1 cm with no oscillation.	Minor sensitivity to floor texture variations.
Corner Navigation	95%	Reliable in-place and recovery turns for L-inside/outside.	Blind spots in front sensor placement caused occasional collisions.
Endurance and Power-Drop	92%	Reliable in-place and recovery turns for L-inside/outside.	Custom robot's higher power draw from 12V motors reduced runtime; slight speed reduction affects responsiveness.

5. Conclusion and Future Scope

This project successfully demonstrates the advantages of integrating PID-based feedback into wall-following navigation, transitioning from a threshold-based open-loop model to a more reliable and accurate closed-loop system. By using ultrasonic sensors to detect wall distances, the robot was able to correct deviations continuously, achieving consistent movement and precise adherence at the setpoint. This implementation validates the core principles of feedback control and highlights the impact of proportional-integral-derivative tuning on overall system performance, where dynamic gain adjustments, such as real-time adaptation of Kp, Ki, and Kd based on environmental feedback or machine learning models, could further optimize responsiveness to varying terrains or speeds, preventing overcorrections in fast segments or sluggish recovery in cluttered areas. The result is a robust and repeatable navigation method suitable for controlled wall-guided environments, laying the foundation for further enhancements, including basic maze-solving capabilities through sequential wall-following patterns that systematically explore dead-ends and backtrack via sensor thresholds. The AlphaBot2-AR platform was also utilized for validating the right-wall following implementation.

Looking ahead, several upgrades can significantly expand the robot's capabilities. Incorporating a 9-DOF Inertial Measurement Unit (IMU) would enable orientation-aware corrections, eliminating residual errors from turns and enhancing path accuracy. Replacing ultrasonic sensors with LiDAR or vision systems could allow finer ranging and complex wall interpretation, supporting adaptive behaviors. Furthermore, integrating forward-facing obstacle sensors and path-planning algorithms like A* would enable dynamic rerouting in cluttered spaces, while extending to full maze-solving by combining wall-following with flood-fill or pledge algorithms for complete labyrinth traversal. These improvements would transform the current prototype into a more intelligent, adaptable, and fully autonomous system, with expanded applications in real-time inventory tracking in dynamic warehouses, automated exploration in search-and-rescue scenarios with variable obstacles, or educational simulations for multi-agent coordination in virtual mazes.

6. GitHub Repository

`https://github.com/aqib-m31/Wall-Following-Robot`